Containers and Kubernetes

Lesson 3 of 12 · Package a model so it runs the same everywhere, and place it on GPUs.

In this lesson → ① What a container is ② Who owns the driver ③ Layer order, measured ④ Bake or mount ⑤ GPUs in Kubernetes ⑥ Sharing a GPU ⑦ Gang scheduling
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 A container is a meal kit

> An **image** is the packed box (code, libraries, settings). A **container** is one running copy. Same box, same dish, any kitchen.

2 The GPU problem: who owns what



the driver can never live inside the image

Piece	Lives in
kernel driver + libcuda.so	**host** (or the GPU Operator driver container)
CUDA runtime libs (cuBLAS, cuDNN, NCCL)	**image** (or PyTorch's pip wheels)
nvcc compiler, headers	build stage only
NVIDIA Container Toolkit	host: connects GPU + driver files at container start

Rule
Never bake the driver into an image.

3 The compatibility rule

> **host driver ≥ minimum driver for the image's CUDA major version**

Image CUDA	Often-quoted minimum driver
12.x	R525 and newer
13.x	R580 and newer

From the notebook (Oct 2026). Check NVIDIA's CUDA Compatibility guide for exact numbers. Not measured here. Script: l03_compat.py. *"No CUDA device found" at start: suspect an old host driver first.*

4 Security: patch the host

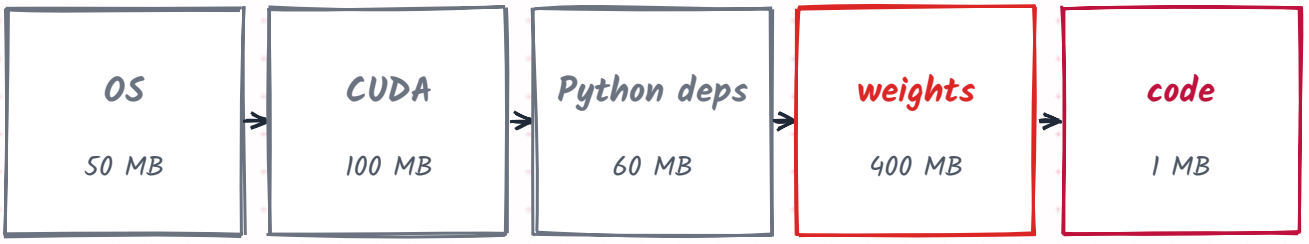
> *CVE-2025-23266 "NVIDIAScape": a container escape through the toolkit's OCI hook, CVSS 9.0. Affected toolkit up to 1.17.7.* **Fixed in 1.17.8** *(GPU Operator 25.3.2). On a shared cluster you run images you did not build, so rebuilding your own images does not help:* **patch the host**.

Image size and reproducibility

- ✓ Layer order: rarely changing first, **code last**
- ✓ Multi-stage build: no compiler in the final image
- ✓ Pin base images by digest, not by tag
- ✓ Commit the lock file; use cache mounts for downloads

5 Layers: the experiment

> A layer's id = **hash(parent layer id + its content)**. Change one layer and every layer above it is made again and shipped again.

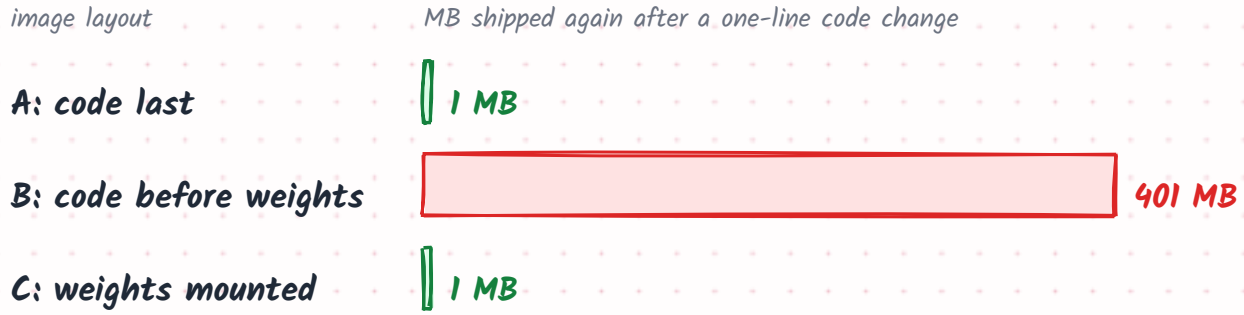


change one layer: every layer to its right is made again

Setup (real bytes, simulated registry)	Value
layers	OS 50 MB, CUDA 100 MB, deps 60 MB, weights 400 MB, code 1 MB
test	push v1, change ONE line of code, push v2, count MB shipped again
registry	a set of known layer ids (Docker was not running here)

Script: l03_layers.py. Hashing the 400 MB layer for real took 0.33 s (1,225 MB/s) on the build machine.

6 Result: layer order



Layout	First push	Change code	New weights
A: weights baked, code last	611 MB	**1 MB**	401 MB
B: code before weights	611 MB	**401 MB**	400 MB
C: weights mounted	211 MB	**1 MB**	**0 MB** in the image

Surprise
The same one-line change shipped **1 MB or 401 MB**, depending only on layer order. Keeping weights out of the image makes both a code change and a model change cheap.

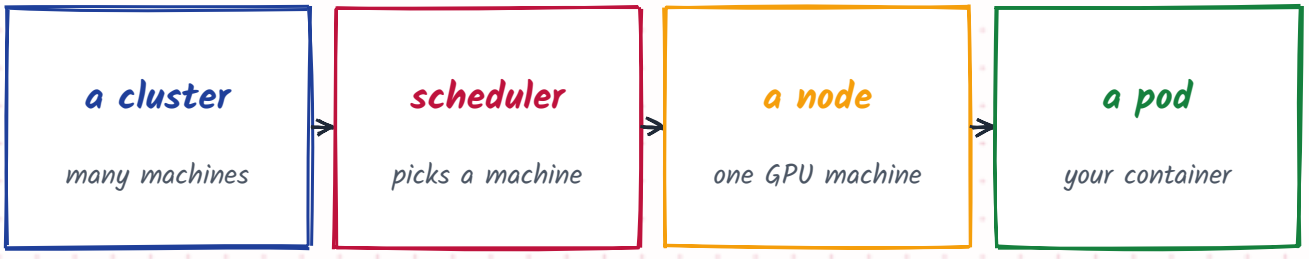
7 Bake or mount the weights?

	Bake into the image	Mount at runtime
best for	small models, under about 1 to 2 GB	multi-GB and TB weights
good	one tidy artifact	update the model without rebuilding the app
bad	every weight change is a huge new layer, slow pulls	one more moving part

Ways to mount

→ **OCI image volume**: weights as their own artifact, mounted read-only (GA in Kubernetes 1.36)
→ a node-local NVMe cache
→ a shared network drive (PVC)
→ object storage through a storage driver
→ pin by digest, like images

8 Kubernetes in one picture



the head chef assigns each dish to a cook

→ **cluster**: all the machines · **node**: one machine · **pod**: one or more containers together
→ the **scheduler** is the head chef: it picks a node for each pod

9 Asking for a GPU

```
resources:
  limits:
    nvidia.com/gpu: 1     # whole numbers, request = limit
```

✓ A **device plugin** on each GPU node announces nvidia.com/gpu
✓ Taint GPU nodes so CPU-only pods stay away; pods that need GPUs carry the toleration
✓ GPU Feature Discovery labels nodes with GPU model and memory
✓ A pod with no GPU request on a GPU node can still see every GPU

10 The GPU Operator

One Helm chart that installs the whole node stack: driver container, container toolkit, device plugin, GPU Feature Discovery, DCGM monitoring and MIG Manager. Versions follow year.month. Latest at the time of the notes: **26.7.1** (23 Sep 2026). If your node image already ships drivers, turn off the operator's driver install.

11 Sharing one GPU



only walls stop a crash next door

Method	Isolation	Use for
MIG	hardware walls (own compute and memory)	several tenants; A100/H100/B200-class, fixed sizes
MPS	memory split equally; no walls	small cooperative inference
time-slicing	none	notebooks and development only

12 DRA: a claim for devices

> Dynamic Resource Allocation: instead of counting nvidia.com/gpu integers, a pod makes a **claim** and a driver offers matching devices. GA since Kubernetes **1.34**.

→ Production use today is mostly **ComputeDomains** (multi-node NVLink on GB200-class racks)

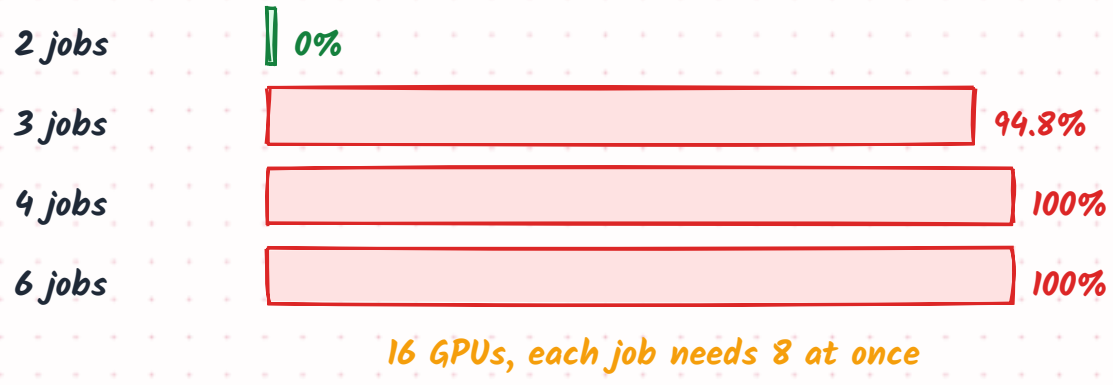
→ For whole-GPU training pods the device plugin is still the safe choice

13 Why gang scheduling?

> Distributed training needs **all N workers** or nobody makes progress. The default scheduler places pods one at a time: job A holds 3 of 4 GPUs, job B holds the 4th, both idle forever. That is a **partial placement deadlock**. Gang scheduling: place all or none.

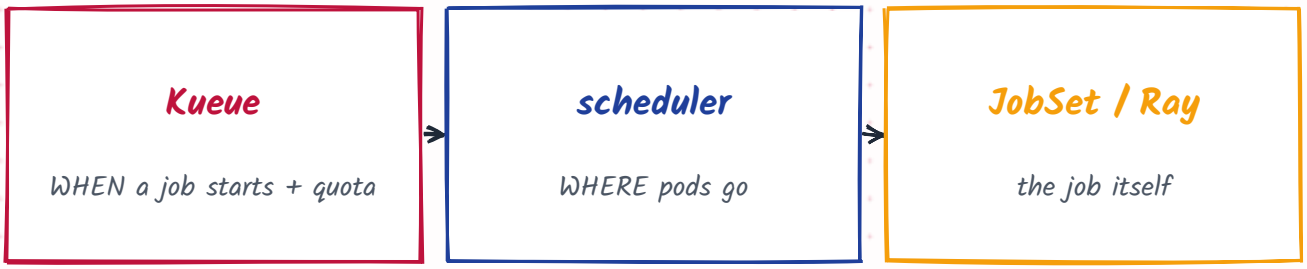
14 The gang simulation

Setting	Value
cluster	16 GPUs
job	8 pods x 1 GPU, makes progress only when all 8 are placed
default scheduler	places pods one by one, in a shuffled arrival order
gang scheduler	places a job only when 8 GPUs are free at once
runs	10,000 random orders per row



A simulation of the idea, not of the real kube-scheduler. Script: l03_gang.py.

15 Queues and gangs in practice



queue first, place second

- → **Kueue**: admission and quota (when a job starts). **Volcano / KAI**: gang-aware schedulers
- → **JobSet, Ray, Kubeflow Trainer**: the job itself
- → Native gang scheduling: beta and **off by default** in Kubernetes 1.37. Do not build production on alpha gates

16 Which orchestrator?

Tool	Good for
Ray	Python-native mixed pipelines (data, train, RL, serve)
Kubeflow Trainer	training jobs on Kubernetes
KServe	serving large models
Slurm	big pretraining; hybrids run Slurm on Kubernetes

17 Plan for failure; a small gotcha

- ✓ Health-check the GPUs
- ✓ Restart the whole gang together when a worker dies
- ✓ Checkpoints make restarts cheap (lesson 2)

```
volumes:
- name: dshm
  emptyDir: {medium: Memory, sizeLimit: 16Gi}  # PyTorch DataLoader workers
```

18 ShopNest Copilot on Kubernetes

Piece	Plan
image	two stages, code on the top layer
weights	mounted, pinned by digest
pod	1 whole GPU, GPU taint toleration, behind a queue
nightly eval	gang job on spot machines (lesson 2)

19 Words to keep

Word	Meaning
Layer	one brick of an image
Mount	attach files at runtime instead of packing them
Taint	a sign that keeps the wrong pods off a node
Gang scheduling	place all of a job's pods together, or none

20 Interview corner

Q. How do you speed up image builds and pushes?

A. Rarely changing layers first, code last, multi-stage builds, big weights outside the image. In my experiment one line of code shipped 1 MB instead of 401 MB.

Q. Why must the host own the GPU driver?

A. A container shares the host kernel and the driver is tied to it, so it cannot be packed in the image. The toolkit connects the host driver files at start.

Q. Why gang scheduling for distributed training?

A. All workers are needed at once. Pod-by-pod placement can deadlock: 3 jobs on 16 GPUs deadlocked 94.8% of the time in my simulation.

Q. MIG, MPS or time-slicing?

A. MIG when tenants must be isolated, MPS for small cooperative inference, time-slicing for notebooks and dev.

21 Interview questions

"Why does layer order in a Dockerfile matter?"

A layer's id is a hash of its parent plus its content, so a change rebuilds and re-ships every layer above it. Rarely changing layers first, code last: one changed line shipped 1 MB instead of 401 MB in my test.

Trap → Copying the whole source tree before installing dependencies.

"Would you bake model weights into the image?"

Only small models, under about 1 to 2 GB. For multi-GB weights, mount them at runtime (OCI image volume, node NVMe cache, PVC or object storage) and pin them by digest. Then a code change and a model change are both cheap.

Trap → A 'latest' tag on weights or images: pin by digest.

"A container says 'no CUDA device found'. What do you check?"

The host driver first: host driver ≥ the minimum for the image's CUDA major version (often quoted: R525 for 12.x, R580 for 13.x). Then the NVIDIA Container Toolkit and the pod's GPU request.

Trap → Installing a driver inside the image. The driver belongs to the host.

"How does a pod get a GPU in Kubernetes?"

A device plugin advertises nvidia.com/gpu and the pod requests it. GPU nodes are tainted so CPU pods stay away; GPU pods carry the toleration. The GPU Operator installs the whole node stack.

Trap → A pod with no GPU request on a GPU node can still see every GPU.

"MIG, MPS or time-slicing?"

MIG gives hardware walls with its own compute and memory, for separate tenants. MPS splits memory with no walls, for small cooperative inference. Time-slicing has no isolation: notebooks and development only.

Trap → Using time-slicing for production multi-tenant serving.

"What is gang scheduling, and when do you need it?"

Place all of a distributed job's pods or none. Pod-by-pod placement can deadlock: in my simulation 3 jobs on 16 GPUs deadlocked 94.8% of the time. Use Kueue with Volcano or KAI.

Trap → Relying on the default scheduler for multi-node training.

22 Your turn

103_layers.py: change layer sizes and order.
103_gang.py: change cluster and job size.
103_compat.py: check a host driver against an image CUDA version.

⚡ Revise in 60 seconds

- → host owns the driver, image owns CUDA libs
- → layer order: **1 MB** vs **401 MB** re-shipped
- → weights outside the image: new model = **0 MB** in the app image
- → whole GPUs only; taints keep CPU pods away
- → MIG = walls, MPS = desks, time-slicing = a chair
- → 3 jobs / 16 GPUs: **94.8%** deadlock, gang **0%**
- → Kueue: when · scheduler: where
- → patch the host (CVE-2025-23266)

Next lesson → CAI-04
Data and storage: where information lives, how to read it fast, and how vectors work

Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**